

Київський національний університет ім. Тараса Шевченка

Кафедра мережесих та інтернет технологій

Лабораторна робота № 6

Дисципліна: Системне програмування

Тема: файлове введення-виведення

Виконав: Студент групи МІТ-41

Поліухович Андрій

Завдання:

1. Створити програму, яка виводить на екран усю ієрархічну структуру певної директорії (файли та папки).

Виконання:

Лістинг коду (Form1.cs):

```
using System;
using System.IO;
using System.Windows.Forms;
using static System.Windows.Forms.VisualStyles.VisualStyleElement;

namespace lab_part2
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            private void buttonOpen_Click(object sender, EventArgs e)
            {
                // Створюємо стандартне діалогове вікно вибору папки
                using (FolderBrowserDialog fbd = new
FolderBrowserDialog())
                {
                    fbd.Description = "Виберіть папку для перегляду її
структури";

                    if (fbd.ShowDialog() == DialogResult.OK)
                    {
                        treeView1.Nodes.Clear();

                        DirectoryInfo rootDir = new
DirectoryInfo(fbd.SelectedPath);
```

```

// Створюємо найперший (головний) вузол у нашому
дереві
TreeNode rootNode = new TreeNode(rootDir.Name);
treeView1.Nodes.Add(rootNode);

// Запускаємо процес малювання всього, що є
всередині
BuildTree(rootDir, rootNode);

// Розгортаємо головну папку одразу після
завантаження
rootNode.Expand();
    }
}

private void BuildTree(DirectoryInfo dirInfo, TreeNode node)
{
    try
    {
        // 1. Отримуємо і додаємо всі файли з поточної папки
        foreach (FileInfo file in dirInfo.GetFiles())
        {
            node.Nodes.Add( file.Name);
        }

        // 2. Отримуємо всі підкаталоги (папки всередині
        папки)
        foreach (DirectoryInfo subDir in
dirInfo.GetDirectories())
        {
            TreeNode subNode = new TreeNode( subDir.Name);
            node.Nodes.Add(subNode);

            // Викликаємо цей самий метод для підпапки
            (Рекурсія!)
            BuildTree(subDir, subNode);
        }
        catch (UnauthorizedAccessException)
        {
            // Якщо це якась системна папка (наприклад,
            Windows), куди не можна заходити
            node.Nodes.Add("[Відмовлено в доступі]");
        }
    }
}

```

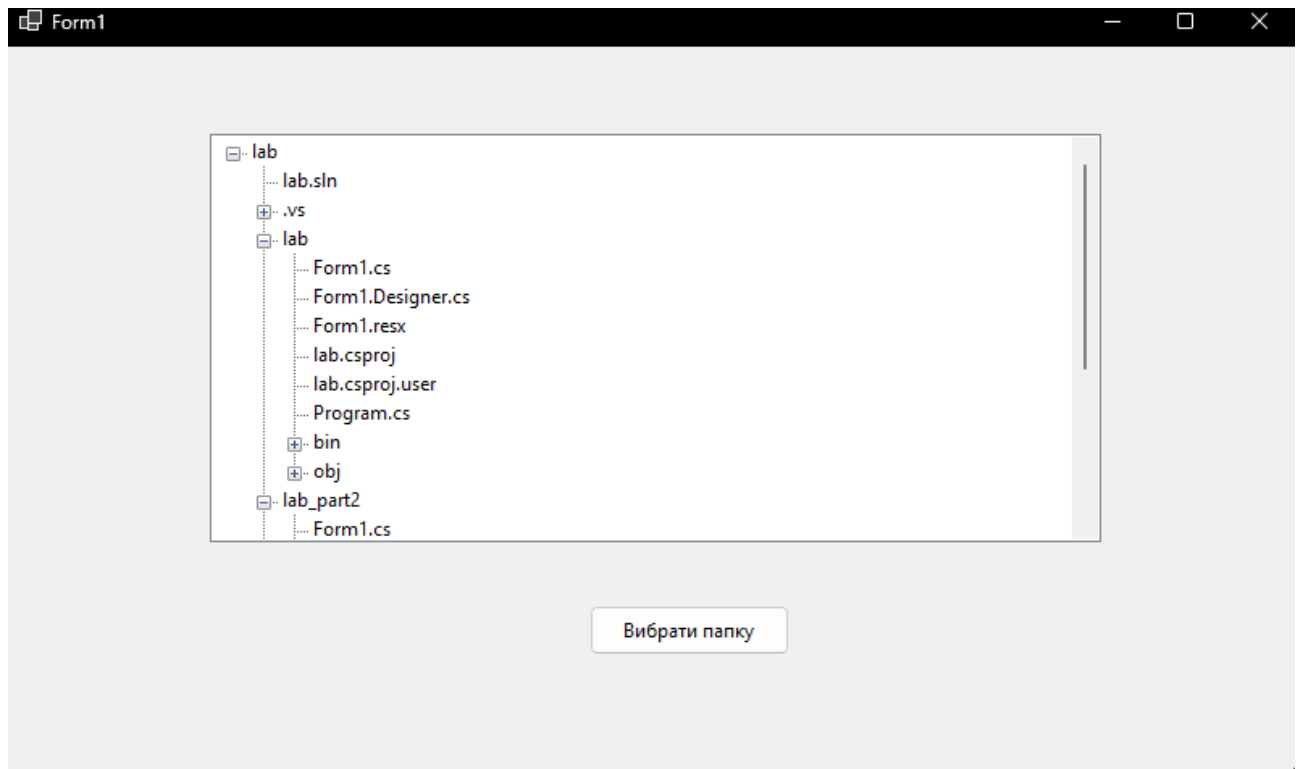


Рисунок 6.1 – Результат роботи програми з завдання 1

На рисунку 6.1 зображено вікно програми після вибору стартової директорії. У компоненті TreeView візуально відображено ієрархічне дерево вкладених папок та файлів.

Аналіз роботи:

У цьому завданні реалізовано графічну програму для виведення ієрархічної структури каталогів за допомогою компонента TreeView. Робота починається із вказання шляху за допомогою FolderBrowserDialog. Після цього створюється об'єкт типу DirectoryInfo. Програма використовує рекурсивний алгоритм: за допомогою методу GetFiles() ми отримуємо всі файли у поточній папці, а за допомогою GetDirectories() — усі підкаталоги, для кожного з яких рекурсивно викликається функція BuildTree. Для уникнення аварійного завершення програми передбачено обробку винятків UnauthorizedAccessException, яка спрацьовує при спробі доступу до захищених системних папок.

Завдання:

2. Створити програму, що виконує пошук заданого файлу та виводить повну інформацію про місце розташування знайдених екземплярів.

Виконання:

Лістинг коду (Form1.cs):

```
using System;  
using System.Drawing;  
using System.IO;  
using System.Windows.Forms;
```

```

namespace lab_part2
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            richTextBoxResults.Font = new Font("Consolas", 10);

            // Налаштування "прозорої" підказки (Placeholder)
            textBoxFileName.Text = "*.txt";
            textBoxFileName.ForeColor = Color.Gray;

            // Прив'язуємо події для зникнення/появи тексту
            textBoxFileName.Enter += RemoveText;
            textBoxFileName.Leave += AddText;
        }

        // Коли користувач клікає на поле – текст зникає, колір стає
чорним
        private void RemoveText(object sender, EventArgs e)
        {
            if (textBoxFileName.Text == "*.txt")
            {
                textBoxFileName.Text = "";
                textBoxFileName.ForeColor = Color.Black;
            }
        }

        // Коли користувач забирає курсор, а поле пусте – повертаємо
сіру підказку
        private void AddText(object sender, EventArgs e)
        {
            if (string.IsNullOrEmpty(textBoxFileName.Text))
            {
                textBoxFileName.Text = "*.txt";
                textBoxFileName.ForeColor = Color.Gray;
            }
        }

        private void buttonSearch_Click(object sender, EventArgs e)
        {
            string fileName = textBoxFileName.Text;

            if (string.IsNullOrEmpty(fileName) || fileName ==
            "*.txt")
            {
                MessageBox.Show("Будь ласка, введіть назву файлу!");
                return;
            }

            richTextBoxResults.Clear();
        }
    }
}

```

```

        richTextBoxResults.AppendText($"🔍 ГЛОБАЛЬНИЙ ПОШУК:
'{fileName}'\n");
        richTextBoxResults.AppendText(new string('-', 65) +
"\n\n");

        // Отримуємо список всіх логічних дисків
        string[] drives = Directory.GetLogicalDrives();
        int totalFound = 0;

        foreach (string drive in drives)
        {
            richTextBoxResults.AppendText($"[ Сканування диска
{drive} ]\n");
            Application.DoEvents();

            DirectoryInfo rootDir = new DirectoryInfo(drive);
            totalFound += SafeSearch(rootDir, fileName);
        }

        richTextBoxResults.AppendText($"📁 ГОТОВО! Знайдено
файлів: {totalFound}\n");
    }

    private int SafeSearch(DirectoryInfo dir, string pattern)
    {
        int count = 0;
        try
        {
            FileInfo[] files = dir.GetFiles(pattern);

            foreach (FileInfo f in files)
            {
                richTextBoxResults.AppendText($"📄
{f.Name}\n");
                richTextBoxResults.AppendText($"├ Папка:
{f.DirectoryName}\n");
                richTextBoxResults.AppendText($"├ Розмір:
{FormatSize(f.Length)}\n");
                richTextBoxResults.AppendText($"├ Створено:
{f.CreationTime:dd.MM.yyyy HH:mm}\n");
                richTextBoxResults.AppendText($"└ Повний
шлях: {f.FullName}\n");
                richTextBoxResults.AppendText(new string('-',
65) + "\n\n");

                count++;
                Application.DoEvents();
            }

            DirectoryInfo[] subDirs = dir.GetDirectories();
            foreach (DirectoryInfo subDir in subDirs)
            {
                count += SafeSearch(subDir, pattern);
            }
        }
    }

```

```

    }
}
catch (UnauthorizedAccessException) { /* Пропускаємо
системні папки */ }
catch (Exception) { /* Ігноруємо інші помилки */ }

return count;
}

private string FormatSize(long bytes)
{
    if (bytes < 1024) return $"{bytes} B";
    if (bytes < 1048576) return $"{(bytes / 1024.0):F1} KB";
    return $"{(bytes / 1048576.0):F2} MB";
}
}
}

```

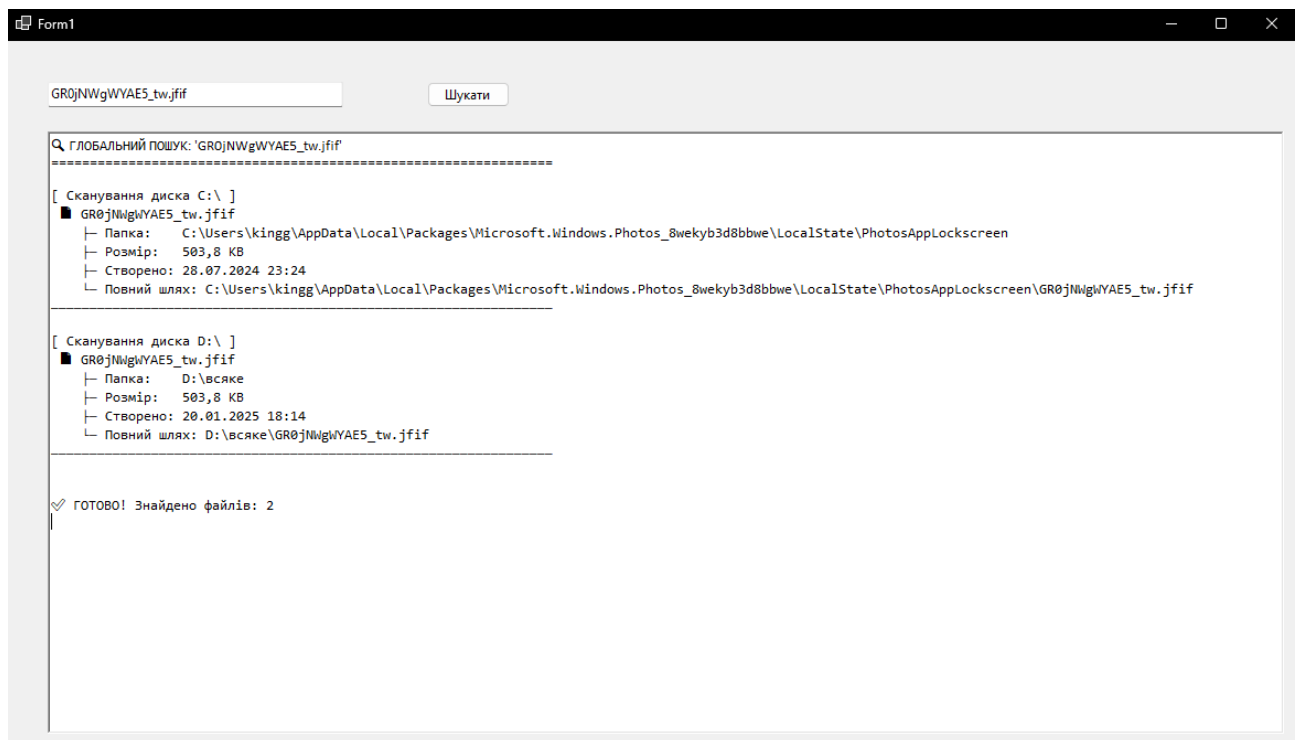


Рисунок 6.2 – Результат роботи програми з завдання 2

Опис результату:

На скріншоті відображено процес глобального пошуку заданого файлу по дисках системи. У текстовому полі виведено детальну інформацію про знайдений екземпляр файлу (розмір, повний шлях, дату створення) та загальну кількість знайдених збігів.

Аналіз роботи:

Розроблена програма виконує глобальний пошук заданого файлу по всіх дисках комп'ютера. Для початку сканування використовується метод `Directory.GetLogicalDrives()`, який повертає масив усіх доступних дискових пристроїв. Алгоритм безпечного пошуку (`SafeSearch`) також працює за

принципом рекурсії: він перевіряє файли за шаблоном через метод GetFiles(pattern), а потім заглиблюється в наступні папки. Завдяки використанню класу FileInfo ми отримуємо детальну інформацію про знайдений екземпляр файлу (його повний шлях, час створення та розмір). Щоб графічний інтерфейс не зависав під час важкого сканування диска, у цикл вбудовано виклик Application.DoEvents().

Завдання:

3. Високий рівень. Створити програму з графічним інтерфейсом, яка б відтворювала основні можливості файлового менеджера.

Виконання:

Лістинг коду (Form1.cs):

```
using System;
using System.IO;
using System.Windows.Forms;

namespace lab_part3
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            // Прив'язка подій
            listBoxContent.DoubleClick += new
EventHandler(listBoxContent_DoubleClick);
            listBoxContent.SelectedIndexChanged += new
EventHandler(listBoxContent_SelectedIndexChanged);
        }

        // --- Заглушки для Дизайнера ---
        private void textBoxCurrentPath_TextChanged(object sender,
EventArgs e) { }
        private void labelFileInfo_Click(object sender, EventArgs e)
{ }

        private void Form1_Load(object sender, EventArgs e) { }
        // -----

        private void buttonOpen_Click(object sender, EventArgs e)
        {
            using (FolderBrowserDialog fbd = new
FolderBrowserDialog())
            {
                if (fbd.ShowDialog() == DialogResult.OK)
                {
                    LoadDirectory(fbd.SelectedPath);
                }
            }
        }
    }
}
```

```

    }

    private void LoadDirectory(string path)
    {
        try
        {
            DirectoryInfo dir = new DirectoryInfo(path);
            if (!dir.Exists) return;

            textBoxCurrentPath.Text = path;
            listBoxContent.Items.Clear();

            if (dir.Parent != null)
            {
                listBoxContent.Items.Add(".. [Бропу]");
            }

            foreach (DirectoryInfo subDir in
dir.GetDirectories())
            {
                listBoxContent.Items.Add("📁 " + subDir.Name);
            }

            foreach (FileInfo file in dir.GetFiles())
            {
                listBoxContent.Items.Add("📄 " + file.Name);
            }
        }
        catch (UnauthorizedAccessException)
        {
            MessageBox.Show("Доступ до цієї папки заборонено.");
        }
    }

    private void listBoxContent_DoubleClick(object sender,
EventArgs e)
    {
        if (listBoxContent.SelectedItem == null) return;

        string selected =
listBoxContent.SelectedItem.ToString();
        string currentPath = textBoxCurrentPath.Text;

        if (selected == ".. [Бропу]")
        {
            DirectoryInfo parent =
Directory.GetParent(currentPath);
            if (parent != null) LoadDirectory(parent.FullName);
            return;
        }

        if (selected.StartsWith("📁 "))
        {

```



```

        string folderName = selected.Replace("📁 ", "");
        string newPath = Path.Combine(currentPath,
folderName);
        LoadDirectory(newPath);
    }
    else if (selected.StartsWith("📄 "))
    {
        ShowFileInfo(selected, currentPath);
    }
}

private void listBoxContent_SelectedIndexChanged(object
sender, EventArgs e)
{
    if (listBoxContent.SelectedItem == null) return;

    string selected =
listBoxContent.SelectedItem.ToString();
    string currentPath = textBoxCurrentPath.Text;

    if (selected.StartsWith("📄 "))
    {
        ShowFileInfo(selected, currentPath);
    }
    else
    {
        labelFileInfo.Text = "Виберіть файл для перегляду
інформації";
    }
}

private void ShowFileInfo(string selectedItem, string
currentPath)
{
    string fileName = selectedItem.Replace("📄 ", "");
    FileInfo file = new FileInfo(Path.Combine(currentPath,
fileName));

    // Написано в один рядок, щоб уникнути помилки "Newline
in constant"
    labelFileInfo.Text = "Файл: " + file.Name + "\nРозмір: "
+ (file.Length / 1024) + " KB\nСтворено: " +
file.CreationTime.ToString("dd.MM.yyyy HH:mm");
}
}
}

```

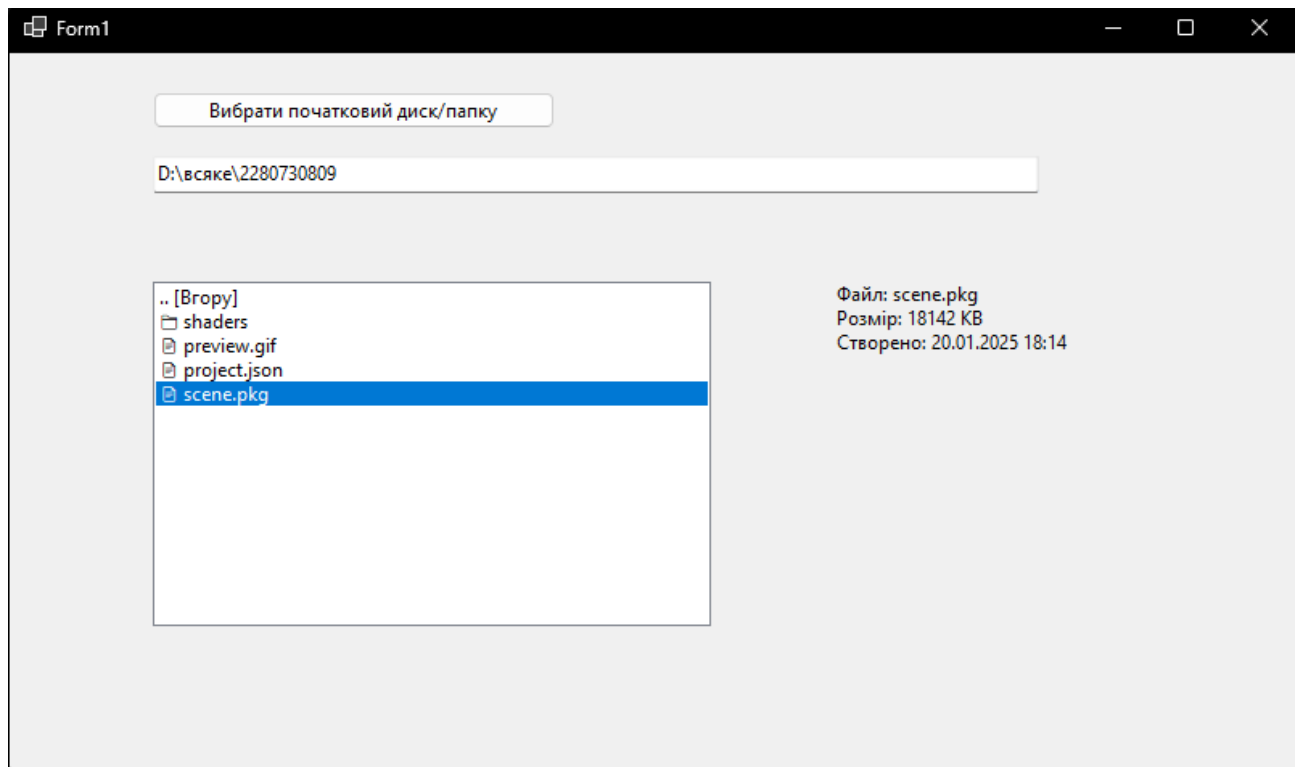


Рисунок 6.3 – Результат роботи програми з завдання 3

Опис результату:

На рисунку 6.3 продемонстровано роботу розробленого файлового менеджера. У центральному списку відображено вміст вибраної директорії з візуальним розділенням на папки та файли, а також кнопкою переходу на рівень вище. Праворуч автоматично виводиться інформація про виділений файл (розмір та час створення).

Аналіз роботи:

У цьому завданні створено графічний додаток, який відтворює базовий функціонал файлового менеджера. Навігація по файловій системі реалізована за рахунок обробки події подвійного кліку мишею `DoubleClick`. Якщо користувач обирає папку, програма формує новий шлях за допомогою методу `Path.Combine` та передає його у метод `LoadDirectory`. Якщо об'єкт є файлом, програма використовує конструктор `FileInfo`, щоб відобразити його розмір та дату створення на екрані.

Висновок:

Під час виконання лабораторної роботи було успішно засвоєно принципи роботи з файловою системою в середовищі .NET. Доведено, що класи `DirectoryInfo` та `FileInfo` забезпечують зручний об'єктно-орієнтований підхід до перегляду структури каталогів та управління файлами. Реалізація рекурсивних алгоритмів дозволила створити ефективні механізми глобального пошуку та побудови дерева каталогів, а використання блоків `try-catch` гарантувало стабільність програм при зустрічі із захищеними областями пам'яті операційної

системи. Створення власного файлового менеджера закріпило навички роботи з графічним інтерфейсом Windows Forms.